

ECE 315

Steven Knudsen, PhD, PEng

Lecture 4 :

- Finish Microprocessor/controller architecture review
- Operating Systems
- PetaLinux
- RTOS introduction
- Assignment #1a and #1b



**UNIVERSITY
OF ALBERTA**

Last Lecture

- Microprocessor/controller architecture review
- Examples relevant to ECE 315

This Lecture

- Finish Microprocessor/controller architecture review
- Operating Systems
- PetaLinux
- RTOS Introduction

Learning Objectives

- Understand the difference between a general operating system and one intended for embedded computing
- Appreciate the difference between hard- and soft-real time operation and constraints
- Identify and understand the main scheduling approaches

Labs and Assignments

- Labs start next week. Please attend your section.
- Assignment #1 (parts a and b) out this week
 - Assignment #1a due on Jan 20th so that you have more background for the labs next week
 - Assignment #1b builds on lectures to date

Assignment #1a – Part 1 FreeRTOS Introduction

- Posted to Canvas
- Start with Wokwi template
- Extend the template to create three (3) tasks in a producer-consumer pattern
 - One producer (master) and two consumers (workers)
 - Workers have different capacities; one can take on more work than the other

Assignment #1b – Elaborate Lecture material

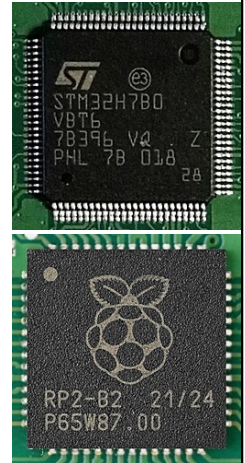
- Posted to Canvas

RP2040 and Cortex-M0+

- Other ARM families are Cortex-R and Cortex-M
 - Support real-time and low-power applications, respectively
- We will consider the RP2040 since it is the processor on the Raspberry Pi Pico
 - The Pico is used in multiple MCTR courses
 - Very inexpensive yet powerful. RP2040 is \$CA1.10 in Qty 1
 - Pico W (WiFi and Bluetooth) is \$CA10.32 in Qty 1
- The ARM architecture it's based on is the Cortex-M0+

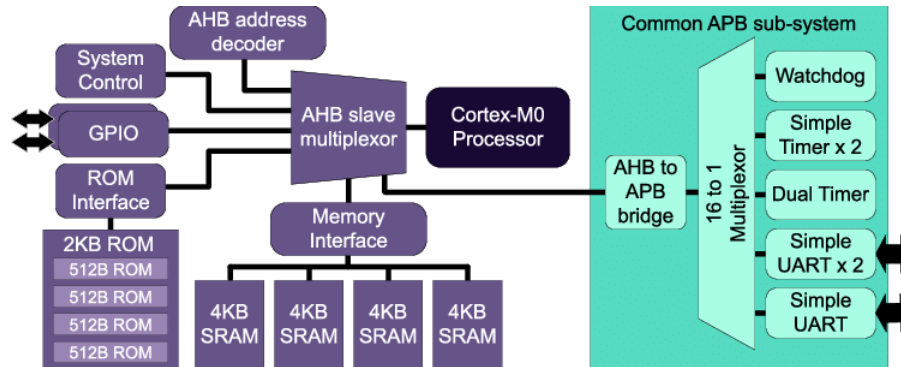
Examples

- We look to a few example that are readily available
- STM32 - a family of 32-bit microcontroller and microprocessor integrated circuits by STMicroelectronics.
- Pico (RP2040, RP2350) - a 32-bit dual-core ARM Cortex-M0+ microcontroller designed by Raspberry Pi Ltd.



Basic Architecture

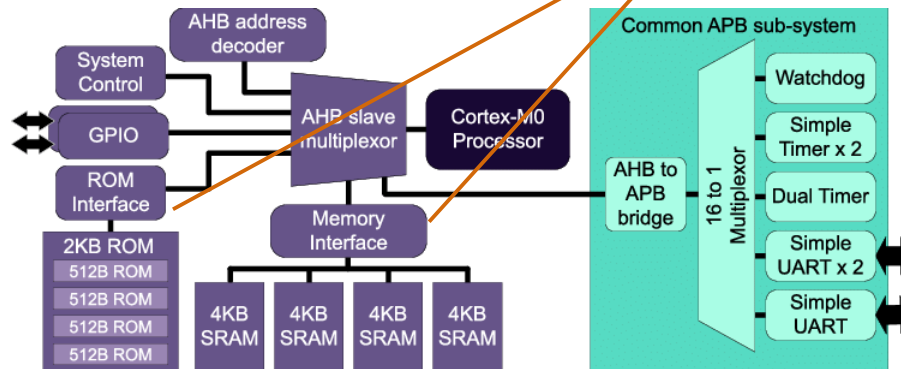
- Advanced high-performance bus (AHB) matrix



Basic Architecture

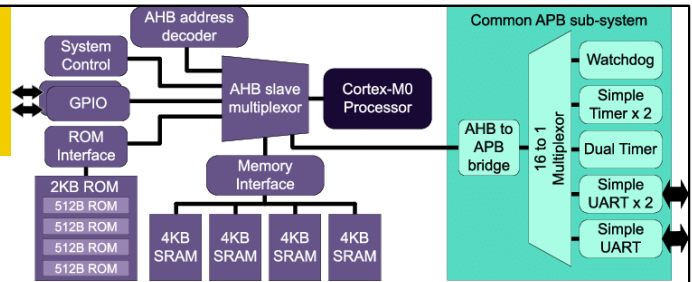
- Advanced high-performance bus (AHB) matrix

Hybrid architecture



Basic Architecture

- **Read-only Memory (ROM)**
 - Manufacturer-specific program and data to support the processor's operations, e.g., specialized embedded devices like analogue-to-digital converters
- **GPIO** – General Purpose Input/Output
 - Typically control logic levels (signal) on external pins
- **SRAM** – Static Random Access Memory
 - low-power RAM with fast access
- **UART** – Universal Asynchronous Receiver-Transmitter
 - Supports serial data communications (send bits on one wire, receive on another)
- **Timer** – Can generate events based on programmed time intervals
- **Watchdog** – Used to monitor the processor or peripherals to make sure they are alive



ARM Cortex-M Family

Cortex-	Description	Examples by ST Micro and Nordic
M0	is the smallest Cortex-M processor, designed for entry-level microcontrollers.	STMf0, rRF51
M0+	is a highly energy-efficient processor, specifically useful in wearables for healthcare and fitness.	STM32[C0,G0,L0,U0,WL,WB0,WB]
M3	has more powerful instruction set, and hence, provides more computation power, specifically useful in smart-home devices.	STM32[F1, F2]
M4	further includes digital signal processing (DSP) instructions and optional integration of a single-precision floating point unit (FPU). These features make Cortex-M4 useful for mixed signal (analog and digital) and control systems.	STM32[F3,F4,G4,H7,L4,L4+,WL,WB]
M7	is a high-performance Cortex-M processor, which also supports cache and external memory. Cortex-M7 has the capabilities of a real-time processor, particularly useful for computationally intensive automation applications.	STM32[F7,H7]
M23, M33, and M35P	include TrustZone security, which protects critical information within a system. Cortex-M35P has additional security features to protect confidential data from several forms of physical security attacks. This helps keeping personal information safe in gadgets like smartwatches and home security systems.	STM32[H5,L5,U3,U5,WBA], nRF[91,5340,54, 54H20]
M52, M55, and M85	processors each implement Helium™ technology that supports vector instructions, making them useful in signal processing and machine-learning applications such as audio processing, power electronics, voice command recognition and still or low frame-rate image processing.	STM32N6



UNIVERSITY
OF ALBERTA

13

Operating Systems



UNIVERSITY
OF ALBERTA

14

Why Operating Systems Now?

I thought this course was about computer interfacing...



Why Operating Systems Now?

I thought this course was about computer interfacing...

- It is, but to do the work on assignments and in labs, we need some knowledge of how to write code to connect to and control interfaces and peripherals
- We can do this with “bare metal” programming and via an operating system

What is an Operating System (OS)?

An OS is software that manages system resources for one or more applications. It takes care of

1. **Process Management** – Creates, schedules, and terminates processes (running programs)
2. **Memory Management** – Allocates and tracks the use of system memory
3. **File System Management** – Organizes and controls access to stored data
4. **Device Management** – Coordinates input/output devices (e.g., keyboard, printer, network)
5. **Security and Access Control** – Protects system resources from unauthorized use.

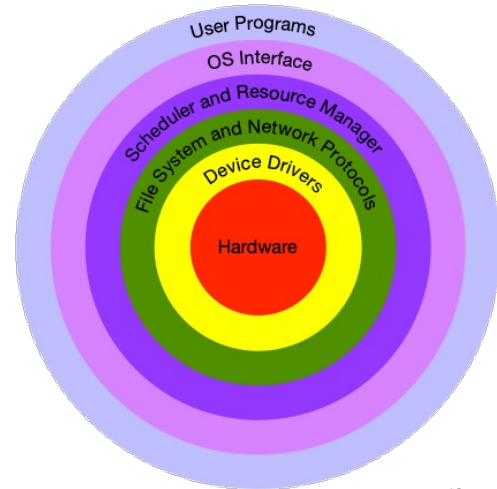
What is OS? Cont'd

In the case of personal computers, tablets, smartphone, servers, and other similar systems, an OS

- Mediates, or acts as an intermediary between users and the hardware,
- Mediates between computer systems that are networked
- In the case of an embedded system, there is usually no user, just other systems, and the OS's role is substantially different
 - We'll dig into that more later...

Layer Model of an OS

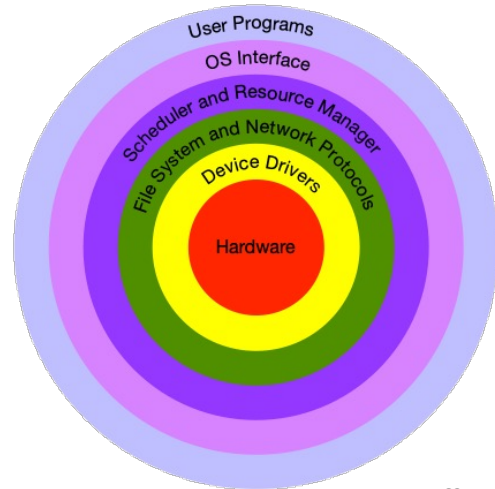
- Starting at the lowest level,
- Hardware can't be changed
- Device drivers are small chunks of code that interact with specific devices, like memory controller, serial interfaces, etc.
- File System and Network Protocols are close to the device drivers for efficiency and performance



We can reuse the diagram from Lecture 3 and focus on the OS

Layer Model of an OS

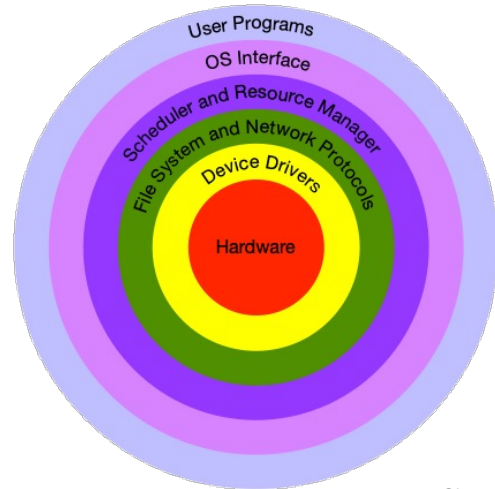
- File System
 - Coordinates the storage and retrieval of data structures.
 - The name “file” is an abstraction, but useful
- Network Protocols
 - Coordinate the transmission and reception of information from other computers and networks
- Both comprise multiple functions organized in libraries



We can reuse the diagram from Lecture 3 and focus on the OS

Layer Model of an OS

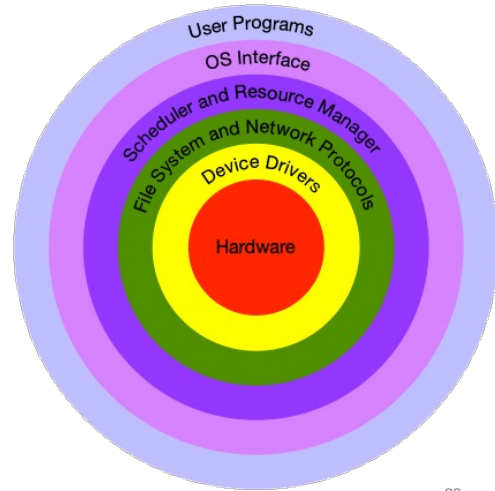
- Scheduler coordinates CPU access by higher layers to OS resources
- It decides which program (aka **Task**) gets CPU time and when
 - Ideally ensures that all tasks get some time
 - Very complex problem on personal or server computer running many tasks concurrently
 - Somewhat simpler for embedded systems, but complicated by hard-real time requirements



We can reuse the diagram from Lecture 3 and focus on the OS

Layer Model of an OS

- Resource manager monitors and manages the system's resources
 - May include the scheduler functions...
 - Decides which tasks get which resources, like memory, file space, and access to input/output devices.
- Monitors resource usage
- Protect against unauthorized access
- Manages resource allocation and deallocation
 - E.g., if a task dies, makes sure resources are released

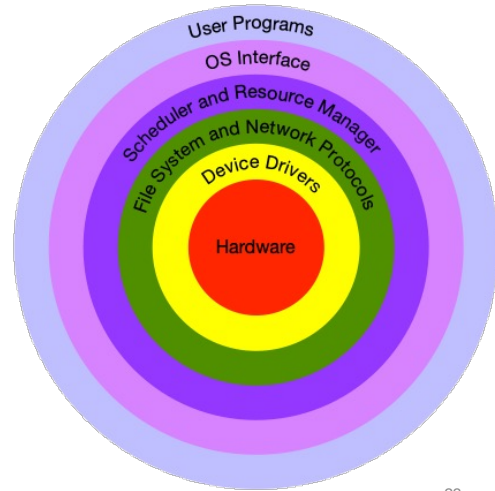


22

We can reuse the diagram from Lecture 3 and focus on the OS

Layer Model of an OS

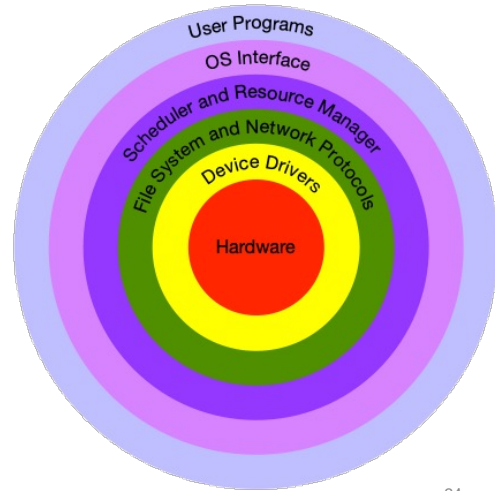
- The OS Interface provides User Programs access to common OS functions
 - CRUD file access
 - Create, Read, Update, Delete
 - Access to peripherals like keyboards, screens, printers, etc.
- User programs are applications that are not native to the system, but are added by users



We can reuse the diagram from Lecture 3 and focus on the OS

Layer Model of an OS

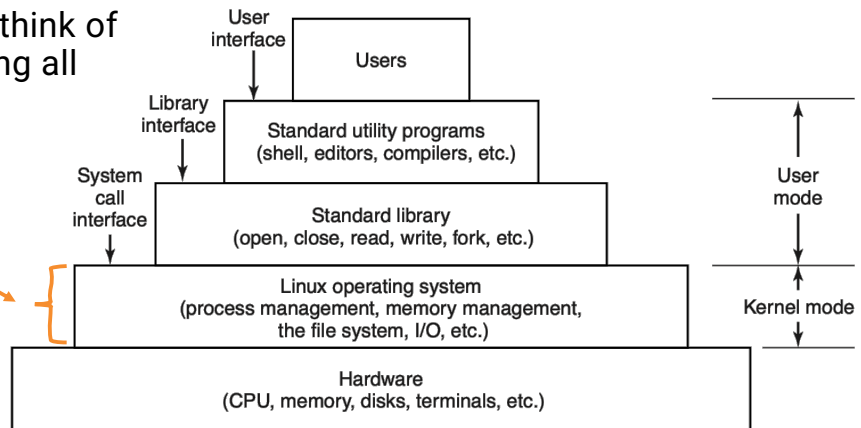
- The lower three layers can be combined into what is called the Kernel
 - Scheduler, Resource Manager, File System, Network Protocols, and common Device Drivers



We can reuse the diagram from Lecture 3 and focus on the OS

Linux OS Layer Model

- We often wrongly think of Linux as comprising all these layers
- It is really only the kernel



Types of Operating Systems

- Batch
- Time-Sharing
- Distributed
- Embedded
- Real-Time

Batch OS

- Similar jobs are collected together and executed as a batch, one after another.
- Users prepare their jobs (programs, input data, etc.) and submit them to the computer operator.
- The OS then processes these jobs sequentially or according to a scheduling order, without manual intervention between them.
- Users do not interact with programs while they run
- Results are returned by the operator to the user when the job completes

Time-sharing OS

- An OS that enables concurrent access to a single computer by dividing CPU time into small intervals called time slices.
- Each active user or process gets a short share of CPU time in turn.
- Users perceive sole use of the computer
- User programs execute in a series of short time intervals, often called slices
- Slices are short enough that users perceive fast responses
 - Humans are way slower than computers
 - A computer can get a lot done in 10 ms... can you?

Distributed OS

- An OS that coordinates and controls multiple interconnected computers (nodes) in a network so they work together as one system.
- Each computer runs its own local OS, but the distributed OS manages communication, resource sharing, and task distribution across all machines.
- Users perceive the system as a single computer
- Big advantage is that it scales; more computers can be added to add capability without a perceived difference in operation

Embedded OS

- An OS specifically optimized to manage hardware and software resources of an embedded computer system.
- It runs on devices that perform specific tasks, often with limited memory, processing power, and energy.
- Unlike general-purpose operating systems (like Windows or Linux desktops), an embedded OS is lightweight, efficient, and tailored for a particular application or device.
- Can be hard- or soft-real time
- Usually no user interface when deployed

Time-Sharing – PetaLinux

- An embedded Linux Software Development Kit (SDK) developed by Xilinx for use with its System-on-a-Chip (SoC) devices, like the Zynq
- Use it to create a customizable Linux-based operating system tailored to your needs
 - Can be very lightweight(!)
- Key Features:
 - Integrated with Xilinx tools like Vivado and Vitis
 - Supports ARM and MicroBlaze architectures
 - Provides board support packages (BSPs) for quick setup
 - Includes tools for kernel, bootloader, and root filesystem configuration



31

Enables rapid prototyping and development of embedded applications

PetaLinux

- At the core of the SDK is a reliance on the Yocto Project

Benefits of Yocto Project



The de facto industry standard "tool kit"

The de facto industry standard "tool kit" for building custom embedded Linux operating systems



The #1 platform to validate new SoC designs

(all architectures) and build BSPs/SBOMs and Reproducible Builds



Preferred platform for a variety of industry initiatives

AGL, RDK Set Top Boxes, TVs, Commercial Switches, Routers, Security Products, Embedded Devices, Medical Devices, and much more

<https://www.yoctoproject.org>

PetaLinux SDK

- Also supports running FreeRTOS, a real-time operating system
- We use it in this course
 - In assignments
 - In labs
- It is an open-source RTOS that is very portable and lightweight
 - Actively maintained and improved
 - Well suited to embedded applications
- More on FreeRTOS in a moment...

Real-Time OS

- In general, an embedded operating system that can meet timing constraints
- Runs multiple programs (aka processes) **seemingly** concurrently
- It manages hardware resources and executes tasks within strict time constraints, ensuring that important operations occur at precise and predictable times

Real-Time OS cont'd

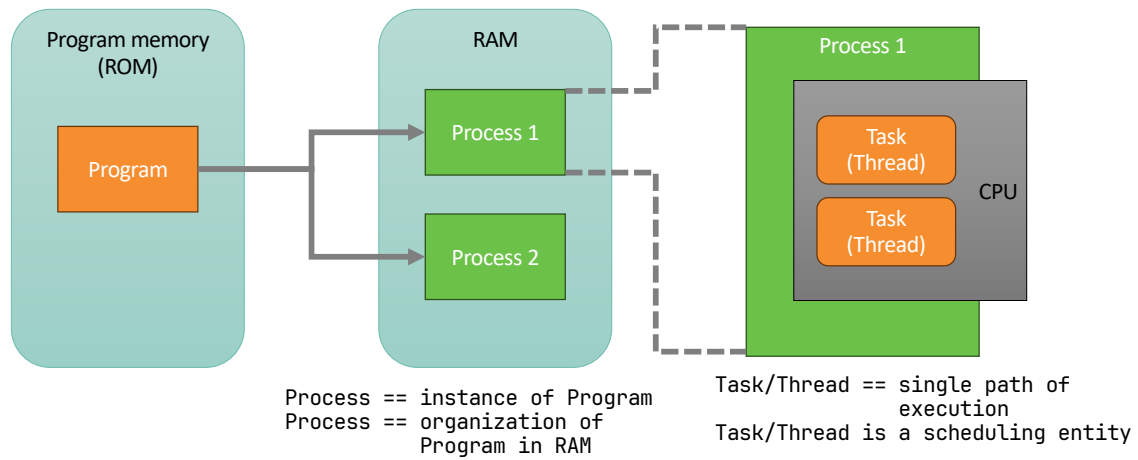
- **Deterministic response:** Guarantees that tasks complete within a fixed time
- **Task prioritization:** High-priority tasks preempt lower-priority ones
- **Minimal latency:** Very short time between event occurrence and response
- **Multitasking support:** Can run multiple tasks simultaneously with precise scheduling
- **Reliability and stability:** Designed to run continuously without failure

Some Useful Concepts and Terms

Processes and Threads

- A program that is executing is a **process**
- A process comprises smaller units called **tasks** or **threads**
- **Tasks** and **threads** are managed via scheduling and synchronization

Processes and Threads cont'd



RAM is mapped onto the CPU in terms of what it accesses.

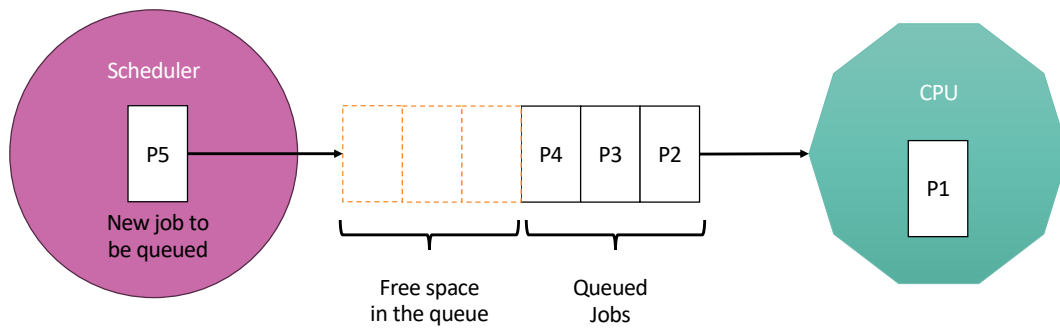
Thread is a subset of instructions and dynamic storage

Scheduling Basics

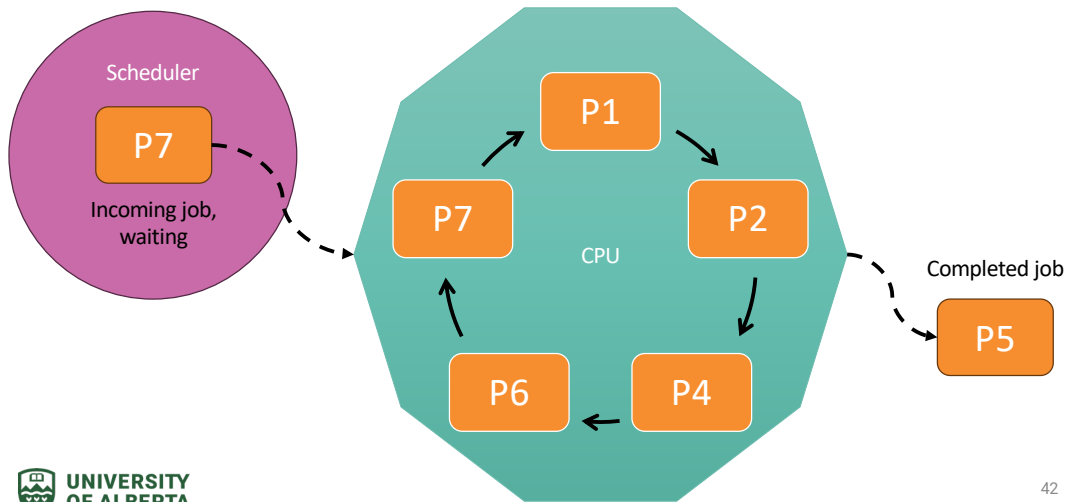
Common algorithms:

- First Come, First Served (FCFS)
 - A non-preemptive scheduling algorithm where the CPU is assigned to processes in the order they arrive in the **ready queue**
 - A process runs to completion before another starts – **sequential execution**
- Round Robin (RR)
 - Each process in the ready queue is given the CPU for a fixed amount of time (called a **time quantum** or **time slice**)
 - If a process does not finish within its time quantum, it is preempted (paused) and placed at the end of the ready queue, allowing the next process to run

First Come, First Serve Scheduler



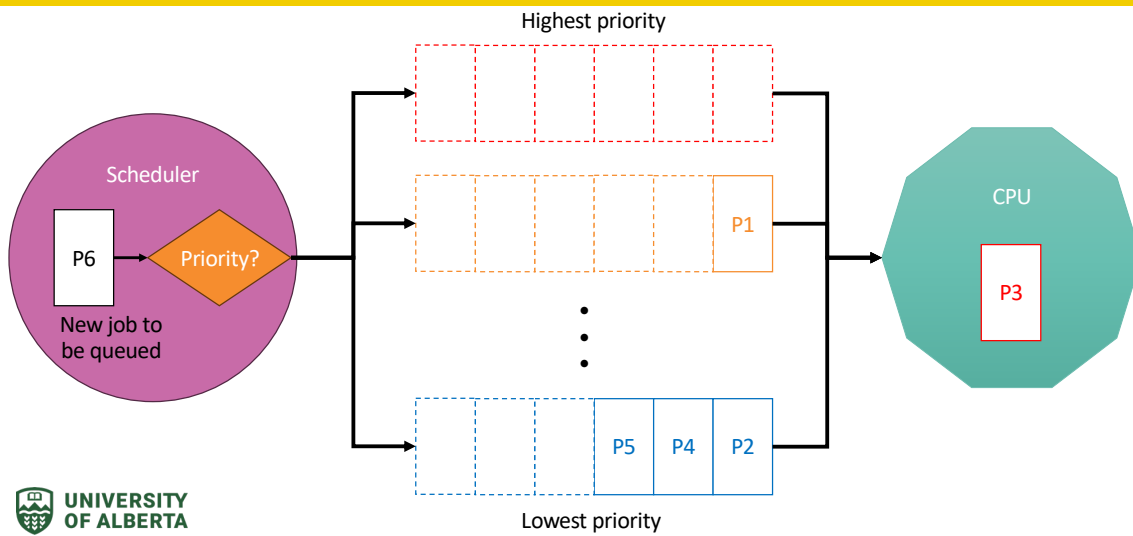
Round-Robin Scheduler



Scheduling Basics cont'd

- Priority Scheduling
 - OS assigns a **priority number** to each process, and the CPU executes processes based on their priority level rather than arrival time
 - A higher priority process is executed before a lower priority one.
 - If two processes have the same priority, they are usually scheduled using First Come First Served (FCFS) order.
 - This algorithm can be either preemptive or non-preemptive.
- Multilevel Queues
 - Two or more task queues that have their own scheduling method are set up
 - A queue is selected using a meta-level scheduling approach and then allowed to manage its tasks

Priority Scheduler



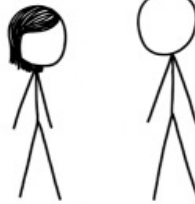
Next Lecture

- More on RTOS concepts via FreeRTOS
- Patterns
- FreeRTOS example showing two patterns
- Assignment #1a and #1b

Questions

SO, DO YOU AND THE OTHER
QA ENGINEERS HAVE ANY FUN
PLANS FOR THE HOLIDAY?

YEAH, ASSUMING THE
SCHEDULING SYSTEM
DOESN'T CRASH.



<https://xkcd.com/7928/>

SOFTWARE TESTING DAY IS A HOLIDAY
CELEBRATED EVERY -1 YEARS ON
JANUARY 0TH AT 25:71 PM.